

NAT

Krystian Sereda 319095

18 maja 2023

1 Wstęp, założenia oraz cel projektu

W ramach tego projektu postanowiłem zasymulować działanie funkcji translacji adresów sieciowych (NAT) z pomocą maszyn wirtualnych oraz zaobserwować proces tłumaczenia adresów sieciowych. Celem projektu jest prezentacja sposobu działania translacji adresów, zrozumienie roli, znaczenia oraz mechanizmu działania NAT w dzisiejszej dobie sieci komputerowych.

Do realizacji tego zadania wykorzystałem narzędzie wirtualizacji VirtualBox, które pozwoliło mi utworzyć dwie maszyny wirtualne. Obie instancje maszyn zostały zainstalowane z systemem Ubuntu 22.04.2 wraz z interfejsem graficznym, co pozwoliło mi na skorzystanie z narzędzia do nasłuchiwania sieci jakim jest Wireshark.

Wireshark jest potężnym narzędziem do analizy ruchu sieciowego, które umożliwia przechwytywanie i analizowanie pakietów przesyłanych przez sieć. Dzięki Wiresharkowi mogłem obserwować, jak pakiety przechodzą przez mój wirtualny router NAT i jakie zmiany zachodzą w adresach sieciowych podczas procesu tłumaczenia.

Plan działania:

- Instalacja VirtualBox.
- Stworzenie dwóch maszyn wirtualnych z systemem Ubuntu 22.04.2 i interfejsem graficznym.
- Konfiguracja sieci na maszynach wirtualnych: ustawienie statycznych adresów IP dla interfejsów wewnętrznych.
- Konfiguracja NAT na VM1.
- Sprawdzenie działania translacji poprzez połączenie się z internetem na VM1 oraz przeprowadzenie testów komunikacji między maszynami wirtualnymi z użyciem ping.
- Obserwacja ruchu sieciowego przy uruchomieniu Wireshark w celu zaobserwowania translacji adresów IP dokonywanej przez NAT.
- Zapisanie wyników testów i analiza obserwacji.

2 Czym jest NAT?

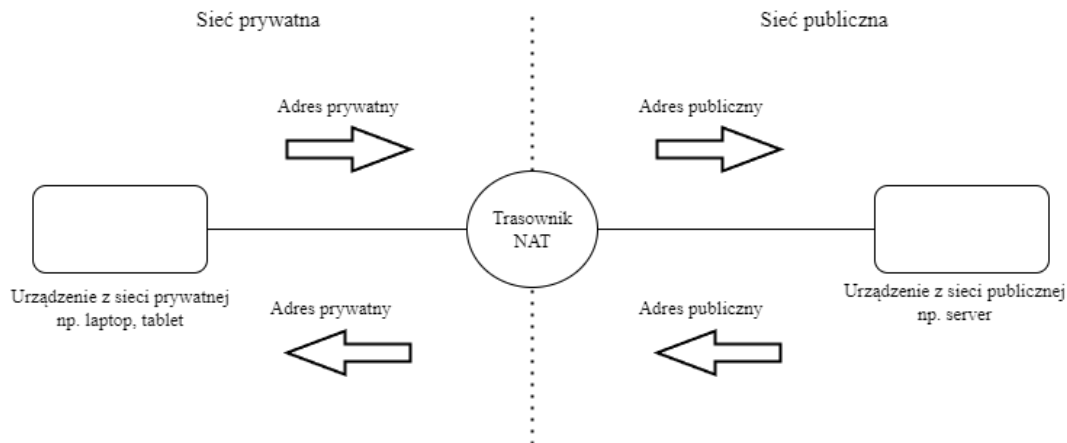
NAT (Network Address Translation) to mechanizm umożliwiający przekładanie adresów sieciowych między różnymi sieciami komputerowymi. Głównym celem NAT jest rozwiązanie problemu braku dostępnych publicznych adresów IP. NAT działa jako pośrednik pomiędzy sieciami prywatnymi a publicznymi, tłumacząc adresy IP i umożliwiając komunikację między nimi.

Dobrym przykładem zastosowania NAT jest jego wykorzystanie w naszych domach.

W domowym środowisku często posiadamy wiele urządzeń, takich jak komputery, telefony, tablety itp. które w celu swobodnego korzystania wymagają stałego podłączenia do internetu. Jednak dostawca usług internetowych często przypisuje nam tylko jeden publiczny adres IP. W tym miejscu NAT znajduje swoje zastosowanie.

W przypadku posiadania urządzenia umożliwiającego translację adresów, takiego jak trasownik, którego zadaniem jest pełnienie roli "tłumacza" adresów IP między naszą siecią prywatną a dostawcą internetu, jesteśmy w stanie korzystać z wielu urządzeń jednocześnie. Zaimplementowany w trasowniku NAT przekłada adresy IP urządzeń w naszej sieci prywatnej na publiczny adres IP.

Kiedy jedno z urządzeń w naszej sieci wysyła żądanie do serwera internetowego, trasownik zamienia adres źródłowy tego żądania na swój publiczny adres IP. Serwer internetowy wysyła odpowiedź na ten adres, a trasownik dokonuje odwrotnej translacji, przekierowując odpowiedź do właściwego urządzenia w sieci prywatnej.



Rysunek 1: Uproszczony schemat zamiany adresów wykorzystywany podczas translacji NAT.

3 Rodzaje NAT

Pojedyncze określenie NAT nie klasyfikuje jednoznacznie implementacji jaka została dokonana. Istnieje kilka sposobów podziału NAT-ów, w zależności od różnych czynników i wymagań sieciowych.

Podział ze względu na typ adresów IP:

- NAT statyczny: Przekształca stały adres IP na inny stały adres IP.
- NAT dynamiczny: Przekształca wiele adresów IP w puli na jeden lub kilka zewnętrznych adresów IP.
- NAT przepełnieniowy: Przekształca wiele adresów IP prywatnych w puli na jeden zewnętrzny adres IP.

Podział ze względu na miejsce implementacji:

- NAT brzegowy: Wykonywany na granicy sieci lokalnej i sieci publicznej.
- NAT wewnętrzny: Wykonywany wewnątrz sieci lokalnej, na przykład w celu umożliwienia komunikacji między różnymi podsieciami wewnętrznymi.

Podział ze względu na skalowalność:

- NAT jednostopniowy: Przekładanie adresów IP odbywa się tylko na jednym urządzeniu.
- NAT wielostopniowy: Przekładanie adresów IP odbywa się na wielu urządzeniach, tworząc hierarchię translacji NAT.

4 NAT - rozwiązanie tymczasowe

NAT od dłuższego czasu pełni istotną rolę w zarządzaniu brakiem dostępnych publicznych adresów IPv4. Jego głównym celem było umożliwienie maskowania wielu adresów IP urządzeń wewnętrznych za jednym adresem publicznym. Jednak z postępującym wprowadzeniem protokołu IPv6, który oferuje nieporównywalnie większą przestrzeń adresową, NAT na skutek tego staje się jedynie rozwiązaniem tymczasowym.

Adresy IPv4 składają się z 32-bitowych liczb binarnych, co umożliwia maksymalnie

4 294 967 296

unikalnych adresów. Jednak z uwagi na rosnące zapotrzebowanie na adresy IP, puli adresów IPv4 szybko się wyczerpała. By temu zapobiec, adresy IPv6 składają się z 128-bitowych liczb binarnych, co daje ogromną przestrzeń adresową wynoszącą

340 282 366 920 938 463 463 374 607 431 768 211 456

Ta znacznie większa ilość dostępnych adresów w protokole IPv6 eliminuje potrzebę stosowania NAT jako rozwiązania maskującego adresy IP.

5 NAT - dodatkowe zabezpieczenie

NAT mimo swojego pierwotnego założenia może również pełnić rolę w zapewnianiu dodatkowego bezpieczeństwa w sieci.

Ukrycie adresów IP sieci wewnętrznej: NAT umożliwia maskowanie prawdziwych adresów IP urządzeń znajdujących się w sieci wewnętrznej (prywatnej) za jednym adresem publicznym.

Filtrowanie pakietów: NAT działa jako punkt kontrolny dla ruchu sieciowego. Może być skonfigurowany do filtrowania pakietów na podstawie adresów źródłowych i docelowych, numerów portów oraz stanu połączenia.

Separacja sieci: NAT pozwala na podział sieci na podsieci, co pomaga w utrzymaniu izolacji urządzeń. Można skonfigurować różne podsieci wewnętrzne z różnymi adresami IP, a NAT zapewni komunikację między nimi.

Ograniczenie dostępu do usług: NAT może być skonfigurowany w taki sposób, aby przekierowywać tylko określone usługi lub porty do odpowiednich urządzeń w sieci wewnętrznej. Pozwala to na ograniczenie dostępu do usług lub zasobów wewnętrznych tylko do uprawnionych urządzeń lub adresów IP.

6 Tworzenie maszyn wirtualnych

Aby przeprowadzić projekt symulujący działanie translacji adresów na maszynach wirtualnych, skorzystałem z darmowego oprogramowania VirtualBox. Poniżej przedstawiam proces tworzenia i konfiguracji maszyn wirtualnych.

Wybór systemu operacyjnego: Jako system operacyjny dla maszyn wirtualnych zdecydowałem się na Ubuntu 22.04.2. Wybór ten opierał się na popularności, stabilności oraz dostępności niezbędnych narzędzi i pakietów.

Tworzenie pierwszej maszyny wirtualnej (VM1):

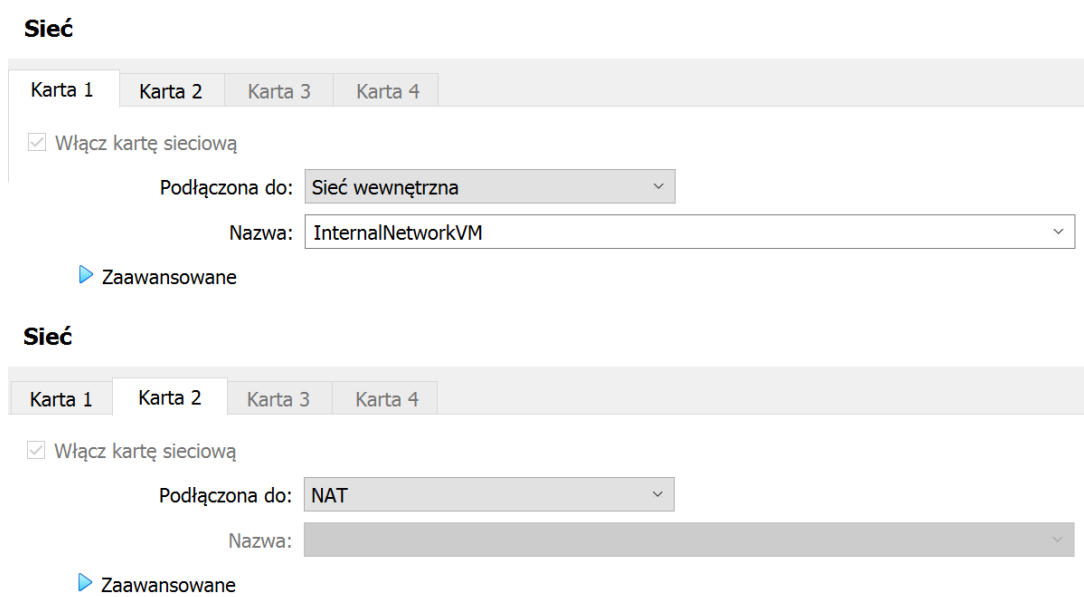
- W VirtualBox utworzyłem nową maszynę wirtualną i przypisałem jej nazwę **VM1** i wcześniej wybraną wersję systemu operacyjnego (Ubuntu 22.04.2).
- Przy konfiguracji maszyny wirtualnej ustawiłem rozmiar pamięci RAM na 4096MB oraz liczbę procesorów na 4 z 12 dostępnych.
- Następnie przystąpiłem do konfiguracji ustawień sieciowych:

- Dodanie dwóch kart sieciowych:
 - * Pierwsza karta (enp0s3) została skonfigurowana jako interfejs wewnętrzny, odpowiedzialny za połączenie z drugą maszyną wirtualną.
 - * Druga karta (enp0s8) została skonfigurowana jako interfejs zewnętrzny, umożliwiający dostęp do internetu. Skonfigurowałem tę kartę w trybie "NAT", który umożliwia maszynie wirtualnej korzystanie z adresu IP gospodarza do komunikacji z internetem.

Tworzenie drugiej maszyny wirtualnej (VM2):

Na czas pobierania potrzebnych narzędzi dodajemy dwie karty sieciowe tak jak w VM1.

- Analogicznie do tworzenia pierwszej maszyny wirtualnej, utworzyłem nową maszynę wirtualną i przypisałem jej nazwę **VM2** oraz obraz ISO z Ubuntu 22.04.2.
- W przeciwieństwie do VM1, druga maszyna wirtualna (VM2) posiada tylko jedną kartę sieciową (enp0s3), którą należy skonfigurować jako interfejs wewnętrzny.



Rysunek 2: Ustawienia sieci maszyn wirtualnych.

7 Konfiguracja maszyn wirtualnych

W celu przeprowadzenia symulacji działania NAT na maszynach wirtualnych, wykonałem szereg konfiguracji. Poniżej przedstawiłem szczegóły dotyczące tych działań w kolejnych etapach.

7.1 Nadanie uprawnień

Aby móc wykonywać zadania wymagające uprawnień superużytkownika: Polecenie `su` - służy do zmiany użytkownika na superużytkownika (root)

`su -`

`sudo adduser [nazwa_użytkownika] sudo`

Dzięki temu użytkownik będzie mógł korzystać z komend `sudo`, umożliwiających wykonywanie poleceń jako superużytkownik.

7.2 Montowanie dysku współdzielonego

Aby udostępnić pliki i skrypty pomiędzy hostem a maszynami wirtualnymi, skorzystano z mechanizmu montowania dysku współdzielonego. W tym celu należy:

1. Wejść w ustawienia maszyny wirtualnej.
2. Przejsć do zakładki *Udostępnione foldery*.
3. Dodać folder, który będzie dostępny dla maszyn wirtualnych.

Po dodaniu folderu, należy użyć polecenia

```
sudo mount -t vboxsf <nazwa_folderu_współdzielonego> <miejsce_docelowe>
```

7.3 Instalacja nftables, net-tools, wireshark

Nftables służy do zarządzania regułami zapory oraz przekazywaniem pakietów w jądrze systemu operacyjnego. Dzięki temu narzędziu możliwe jest przekierowywanie ruchu sieciowego w dalszej części projektu. Wybrałem nftables, ponieważ jest to nowsza alternatywa dla iptables. Mimo to możliwe jest również użycie iptables.

```
sudo apt install nftables
sudo apt install net-tools
sudo apt install wireshark
sudo usermod -aG wireshark <nazwa_uzytkownika>
sudo reboot
```

7.4 Konwersja formatu zakończeń linii skryptów

W przypadku przenoszenia skryptów z gospodarza do maszyn wirtualnych, może występować różnica w formacie zakończeń linii. Na przykład, pliki skryptów mogą posiadać format zakończeń linii Windows (CRLF), który może powodować problemy przy wykonywaniu skryptów na maszynie wirtualnej. Aby rozwiązać ten problem, można skorzystać z narzędzia `dos2unix`, które konwertuje format zakończeń linii na format Unix (LF).

```
sudo apt-get install dos2unix
dos2unix <nazwa_skryptu>
```

7.5 Nadanie uprawnień wykonania skryptom

```
chmod +x <nazwa_skryptu>
```

Dzięki temu skrypt będzie mógł być uruchomiony jako plik wykonywalny.

7.6 Wykorzystane skrypty

Poniżej przedstawiono skrypty użyte w procesie konfiguracji maszyn wirtualnych:

```

test1@test1:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.1 netmask 255.255.255.0 broadcast 0.0.0.0
    ether 08:00:27:71:54:1a txqueuelen 1000 (Ethernet)
    RX packets 492 bytes 73888 (73.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 200 bytes 25170 (25.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

enp0s8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.3.15 netmask 255.255.255.0 broadcast 10.0.3.255
    inet6 fe80::a00:27ff:fe4f:fead prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:4f:fe:ad txqueuelen 1000 (Ethernet)
    RX packets 2376 bytes 2934500 (2.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1291 bytes 114993 (114.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Rysunek 3: Wynik polecenia ifconfig dla VM1.

Skrypt dla maszyny VM1:

```

#!/bin/sh

if [ $# -ne 2 ]; then
echo "Użycie: $0 <nazwa_LAN> <nazwa_WAN>"
exit 1
fi

LAN=$1
WAN=$2

# Ustawienie adresu IP i maski podsieci dla interfejsu LAN
ip address add 192.168.1.1/24 dev $LAN
ip link set $LAN up

# Zapisanie istniejących reguł do pliku (kopia zapasowa)
nft list ruleset > existing_ruleset.nft

# Usunięcie istniejących reguł nftables
nft flush ruleset

# Włączenie przekazywania pakietów w jądrze
sysctl net.ipv4.ip_forward=1

# Ustawienie maskarady na interfejsie WAN
nft add table nat
nft add chain nat postrouting { type nat hook postrouting priority 0 \; }
nft add rule nat postrouting oifname $WAN masquerade

```

Skrypt dla maszyny VM2:

```

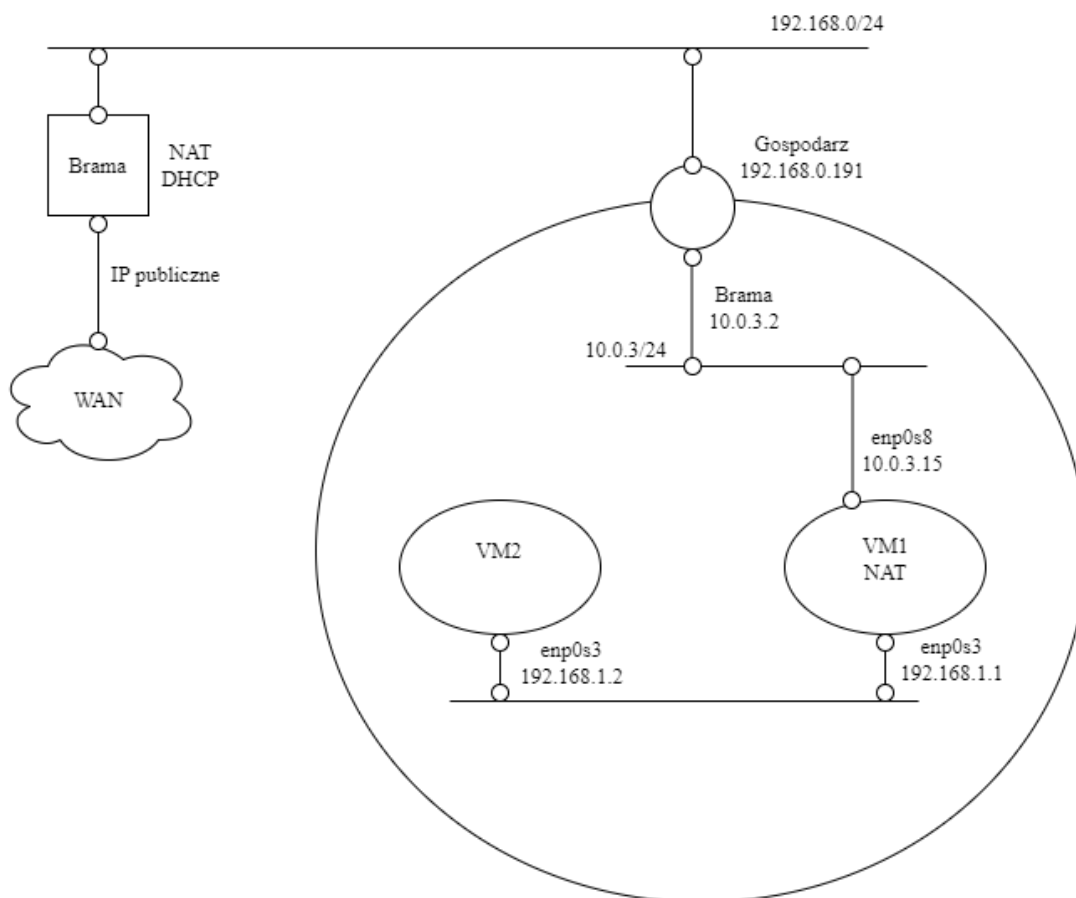
#!/bin/sh

# Ustawienie adresu IP i maski podsieci dla enp0s3 na VM2
ip address add 192.168.1.2/24 dev enp0s3
ip link set enp0s3 up

```

```
# Ustawienie VM1 jako domyślny brama dla VM2
ip route add default via 192.168.1.1
```

8 Symulacja NAT



Rysunek 4: Diagram przedstawiający uproszczony model wykorzystanej w symulacji sieci.

W ramach naszej symulacji działania mechanizmu Network Address Translation (NAT), postanowiłem przetestować komunikację między dwiema maszynami wirtualnymi - VM1 i VM2. Chciałem sprawdzić, jak NAT wpływa na przekierowywanie pakietów i maskowanie adresów.

W pierwszej kolejności po zainstalowaniu wymaganych narzędzi, należy uruchomić wyżej wspomniane skrypty odpowiednio na każdej z maszyn.

Następnie, na maszynie VM2 wykonałem polecenie ping, wysyłając pakiety na adres IP 8.8.8.8.

VM2\$: ping 8.8.8.8

Source	Destination	Protocol	Length	Info
192.168.1.2	8.8.8.8	ICMP	98	Echo (ping) request
8.8.8.8	192.168.1.2	ICMP	98	Echo (ping) reply

Rysunek 5: Podgląd nasłuchiwania sieci z użyciem Wireshark na VM2.

Po czym, skoncentrowałem się na obserwacji ruchu w sieci przy użyciu narzędzia Wireshark. Na drugiej maszynie odczytywałem ruch na interfejsie wychodzącym (enp0s3) oraz na pierwszej oba interfejsy, będąc świadkiem translacji przychodzących adresów przed dalszym wysłaniem ich na interfejs zewnętrzny (enp0s8).

VM2 (192.168.1.2) -> IPpub (10.0.3.15)

IPpub (10.0.3.15) -> VM2 (192.168.1.2)

Source	Destination	Protocol	Length	Info
10.0.3.15	8.8.8.8	ICMP	98	Echo (ping) request
192.168.1.2	8.8.8.8	ICMP	98	Echo (ping) request
8.8.8.8	192.168.1.2	ICMP	98	Echo (ping) reply
8.8.8.8	10.0.3.15	ICMP	98	Echo (ping) reply

Rysunek 6: Podgląd nasłuchiwania sieci z użyciem Wireshark na VM1.

Korzystając z Wiresharka, miałem możliwość zobaczenia, jak adresy źródłowe pakietów z VM2 zostały zastąpione adresem IP maszyny VM1. Dzięki temu, pakiety były w stanie dotrzeć do swoich docelowych maszyn, a odpowiedzi na nie były przekierowywane z powrotem do VM1.